

第二十一届全国大学生
智能汽车竞赛

技 术 报 告

学 校：哈尔滨工程大学

队伍名称：济海追风 2 队

参赛队员：廖旺 易思妍 曾睿宁

带队教师：张立伟

关于技术报告和研究论文使用授权的说明

本人完全了解全国大学生智能汽车竞赛有关保留、使用技术报告和研究论文的规定，即：参赛作品著作权归参赛者本人，比赛组委会可以在相关主页上收录并公开参赛作品的设计方案、技术报告以及参赛模型车的视频、图像资料并将相关内容编纂收录在组委会出版论文集中。

参赛队员签名：

曾睿宇 廖旺 号长妍

带队教师签名：

张之伟

日

期：

2006/8/18

摘 要

本报告记录一台轮腿穿越智能车的设计与调试过程。车辆以博思威龙 V 车模为基础，采用 CYT4BB 双核主控制器和 CYT2BL3 双路无刷电机驱动板。两台 BDS3620 电机负责双轮平衡、行驶和差速转向，四个 BDS300 舵机调节腿高与车体姿态。CM7_0 运行姿态解算、速度、平衡、转向、轮腿和导航程序，CM7_1 负责 MT9V03X 图像采集和四类地形元素处理；两核通过带帧序号与配置序号的共享结果区交换数据。控制器采用速度、直立角度和俯仰角速度串级结构，转向目标由 INS 曲率、元素视觉和地形状态统一仲裁。当前 P3 分支按 0.2 m 间隔记录积分轨迹，并保存元素入口、恢复点和航向等事件信息，复跑时使用动态前视、车体系曲率和出口重锚定。视觉程序分别提取单边桥双边线、颠簸横条序列、坡道侧边线和人工草地边界，报告中的处理图直接由当前 C 代码回放生成。由于尚无统一的正式测试记录，文中不填写识别率、通过率和平均速度等数值。

关键词：轮腿智能车；CYT4BB；双轮平衡；双核控制；复杂地形

目 录

第一章 绪论.....	1
1.1 比赛任务.....	1
1.2 总体思路.....	1
1.3 主要工作.....	1
第二章 系统总体方案.....	3
2.1 整车组成.....	3
2.2 信息流与控制闭环.....	4
2.3 分级供电方案.....	5
2.4 运行状态与安全策略.....	5
第三章 车模与轮腿机械结构.....	7
3.1 车体结构.....	7
3.2 整车主要参数.....	8
3.3 轮腿运动学参数.....	9
3.4 机械结构改造.....	9
3.5 装配与布置原则.....	11
第四章 硬件系统设计.....	13
4.1 硬件模块划分.....	13
4.2 主控拓展主板.....	13
4.3 四路舵机驱动板.....	14
4.4 CYT2BL3 双电机驱动板.....	15
4.5 磁编码器板与信号转接.....	16
4.6 功耗与滤波电容记录.....	18

第五章 软件系统与通信设计.....	19
5.1 开发环境与基础工程.....	19
5.2 双核任务划分.....	19
5.3 初始化流程与实时调度.....	20
5.4 外设通信接口.....	21
第六章 运动控制与轨迹复跑.....	23
6.1 双轮平衡串级控制.....	23
6.2 INS 录点、轨迹复跑与动态前视.....	24
6.3 安全保护.....	26
第七章 感知、视觉与导航.....	27
7.1 传感器与图像输入.....	27
7.2 单边桥引导区处理.....	27
7.3 颠簸路段处理.....	28
7.4 坡道处理.....	29
7.5 人工草地处理.....	30
7.6 观测发布与导航配合.....	33
第八章 整车调试与元素验证.....	35
8.1 离线回放检查.....	35
8.2 板端接口与时序.....	35
8.3 单边桥与坡道联调.....	36
8.4 颠簸与人工草地联调.....	36
附录 A 主要参数.....	37
附录 B 原理图索引.....	38
附录 C 核心算法程序.....	39

C.1 双轮控制 PID 程序.....	39
C.2 单边桥引导区识别程序.....	41
C.3 颠簸路段识别入口.....	43
C.4 坡道侧边线识别入口.....	46
C.5 人工草地识别入口.....	50
C.6 视觉方向误差融合程序.....	51

第一章 绪论

1.1 比赛任务

轮腿穿越组属于室外赛道组。公开科目包括行进绕桩、定点排雷和颠簸路段。行进绕桩要求车辆直行到达掉头区，返程时穿过锥桶间隙；定点排雷要求车辆进入约 1 m 见方的白色方框，在框内旋转至少两周且车轮不能越界；颠簸路段由交错单边桥、颠簸结构、人工草地、三级台阶和坡道组成。^[1-2]

这些任务同时考验车辆的动态平衡、定位、转向和轮腿姿态控制。普通轮式车模只需要处理赛道方向，轮腿车还要在地形变化时调整腿高、横滚和俯仰姿态，因此控制任务之间的配合是整车设计的重点。

1.2 总体思路

本车采用博思威龙 V 车模。CYT4BB 作为整车主控制器，两个 CM7 内核分别运行运动控制和图像处理；CYT2BL3 负责两台无刷电机的三相 PWM 输出；四路舵机驱动板为腿部舵机提供 7 V 电源。IMU、轮速和磁编码器用于平衡与积分导航，GNSS 接口用于室外数据采集和备用定位，摄像头和 DL1B 用于地形判断。

1.3 主要工作

- 完成主控、双电机驱动、四路舵机供电和磁编码器板的连接与调试；
- 完成双轮平衡、速度、差速转向、腿高和横滚控制程序；
- 完成 CYT4BB 双核任务划分和共享内存通信；
- 完成单边桥、颠簸、人工草地和坡道的元素视觉处理，以及台阶距离触发程序；
- 完成 INS 录点、Flash 保存、轨迹复跑和基本安全保护。

第二章 系统总体方案

2.1 整车组成

整车由车体与轮腿机构、主控制系统、双路无刷电机驱动系统、四路舵机驱动系统、多源传感系统、通信与人机交互系统以及 3S 锂聚合物电池组成。主控平台为英飞凌 CYT4BB 核心板及自研拓展板；电机控制器采用 CYT2BL3 双路驱动板；运动执行器为 2 台 BDS3620 无刷电机和 4 个 BDS300 舵机。

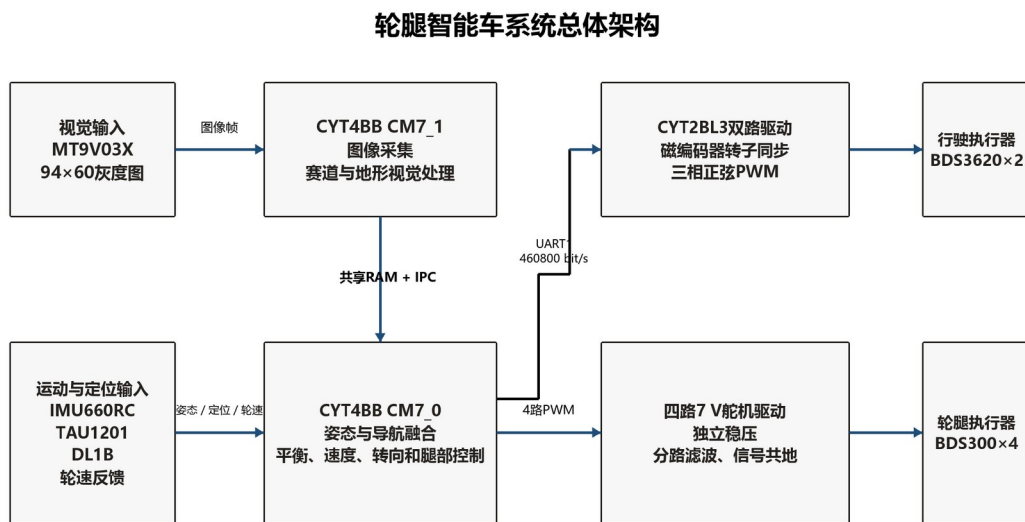


图 2.1 轮腿智能车系统总体架构

第二十一届全国大学生智能汽车竞赛技术报告

表 2.1 整车系统级组成

类别	型号或名称	数量	主要作用
车模平台	博思威龙 V	1	轮腿车机械基础
主控制器	CYT4BB 核心板及主扩展板	1 套	双核运动控制与视觉协同
电机控制器	CYT2BL3 双路驱动板	1	双无刷电机转子同步与三相 PWM
驱动电机	BDS3620	2	行驶、平衡与差速转向
腿部舵机	BDS300	4	轮腿姿态和腿高调节
电池	格氏 3S 1800 mAh 45C	1	整车动力电源
惯性传感器	IMU660RC	1	角速度与加速度测量
摄像头	MT9V03X	1	赛道边界与地形视觉
卫星定位	TAU1201	1	GNSS 数据采集与备用定位
激光测距	DL1B	1	距离触发与地形辅助
磁编码器	MT6816	2	电机转子绝对角度与轮速计算
显示器	TFT180	1	菜单、参数和状态显示
遥控接收机	SBUS 接收机	1	调试与人工接管

2.2 信息流与控制闭环

摄像头、IMU、DL1B 和轮速等信息送入 CYT4BB。CM7_1 将分模式图像结果通过共享 RAM 和 IPC 发送至 CM7_0；CM7_0 在当前 P3 分支中以轮速和姿态积分坐标进行录点与复跑，同时保留 GNSS 数据接口。主核根据轨迹事件、姿态和元素观测生成电机目标与舵机目标。电机目标经 UART1 发送至 CYT2BL3，舵机目标以四路 PWM 输出至舵机驱动板。磁编码器通过两路硬件 SPI 向 CYT2BL3 提供绝对转子角度，驱动程序根据连续角度差估计转速并返回主控制器，构成车体速度与姿态闭环。

2.3 分级供电方案

整车使用一块格氏 3S 锂聚合物电池，容量 1800 mAh，放电倍率 45C。电池母线分别送入舵机驱动板、主控拓展板和 CYT2BL3 电机驱动板，并不是由三块板依次串接。舵机驱动板经输入保护后形成四路独立 7 V 输出；主控拓展板生成系统 5 V、3.3 V 以及 GNSS、摄像头支路；电机驱动板为功率级、低压控制电路和磁编码器供电。常见 3S 电池的标称电压为 11.1 V、满电电压为 12.6 V，这两个数值只用于说明电池类型，不作为本车实测结果。

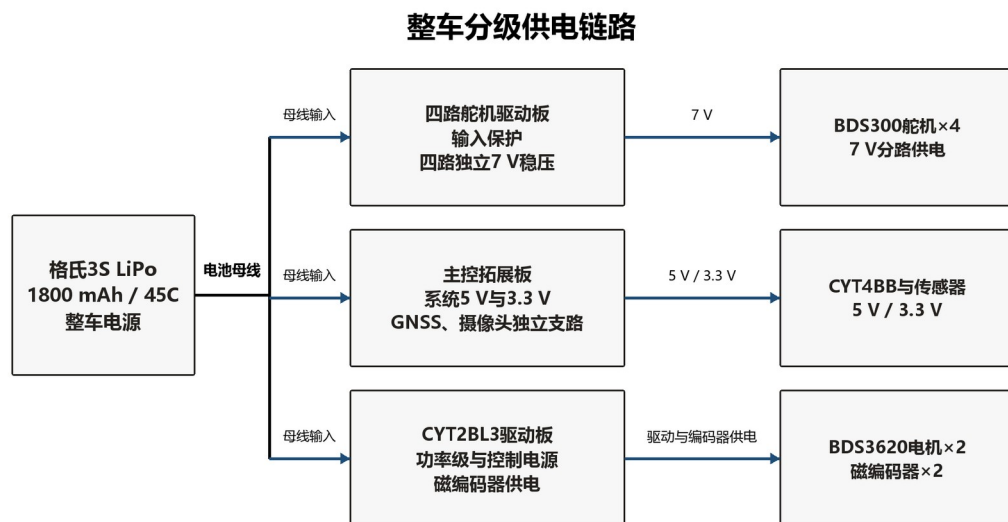


图 2.2 整车分级供电链路

2.4 运行状态与安全策略

系统采用模式与元素状态共同约束控制输出。正常状态下，速度目标、转向目标和腿高目标由任务流程给定；进入特殊地形后，系统按对应状态机调整视觉使用方式、目标腿高、速度工程量和姿态控制。倾倒、离地、轮速异常或跳跃过程超出安全条件时，保护逻辑限制或撤销驱动输出。SBUS 接收机提供调试阶段的人工控制入口，TFT180、按键与 Flash 参数系统用于模式选择、标定和状态检查。

第三章 车模与轮腿机械结构

3.1 车体结构

博思威龙 V 车模采用单轴双轮轮腿结构。两侧车轮既承担行驶和动态平衡，也通过左右差速产生偏航运动；四个舵机驱动左右腿部关节，用于改变车体高度、俯仰与横滚姿态。摄像头安装于车体上方，以获得前方赛道视野；主控板驱动板、电池和传感器沿车体纵向布置，以兼顾质心位置、走线长度与维护便利性。

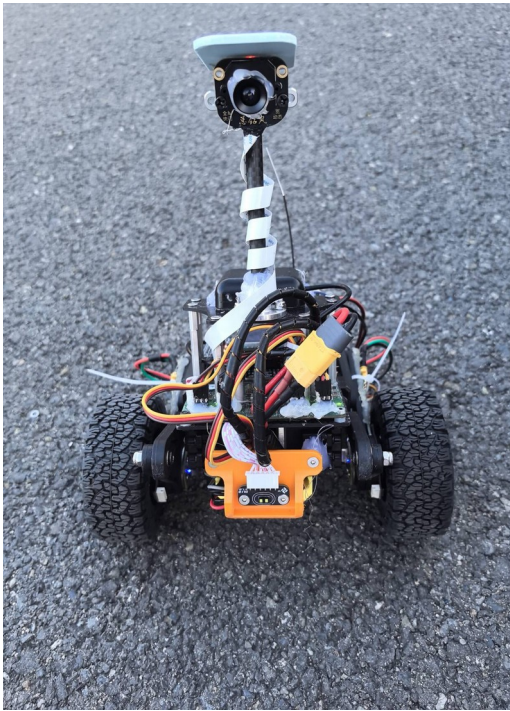


图 3.1 整车前视图

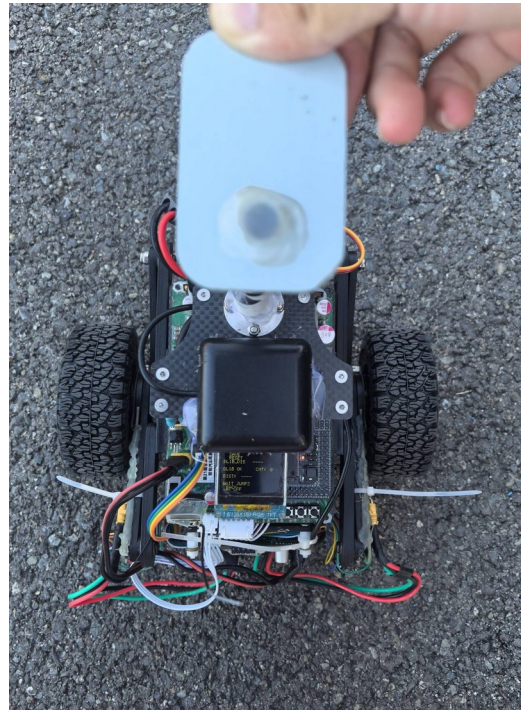


图 3.2 整车俯视图

3.2 整车主要参数

表 3.1 整车机械参数

参数	数值	说明
整车长度	213 mm	车体外廓尺寸
整车宽度	185 mm	车体外廓尺寸
整车高度	270 mm	含上部摄像头安装结构
整车质量	1.35 kg	包含电池
车轮直径	90 mm	左右驱动轮
轮距	175 mm	左右轮中心距
传统前后轴距	不适用	单轴双轮结构
大腿连杆物理长度	70 mm	零件实物总长
小腿连杆物理长度	90 mm	零件实物总长
编码器与磁铁间距	2 mm	安装间隙

3.3 轮腿运动学参数

轮腿逆运动学计算使用的等效参数为 $L_1=37.5\text{ mm}$ 、 $L_2=60.0\text{ mm}$ 、 $L_3=90.25\text{ mm}$ 。这里的参数对应程序中的几何等效量或铰点中心距离，不与 70 mm 、 90 mm 的零件物理总长混用。控制程序根据目标腿高和姿态计算舵机目标角度，并在允许范围内完成关节映射。

$$L_1 = 37.5\text{ mm}, L_2 = 60.0\text{ mm}, L_3 = 90.25\text{ mm} \quad \text{公式 1}$$

式中参数为代码运动学等效量，而非零件外形总长。

基础腿高设置为 45 mm ，可调范围为 $35\sim 100\text{ mm}$ 。普通行驶时使用基础腿高保持质心和控制余量；遇到桥面、台阶、坡道或颠簸时，元素状态机改变目标腿高或横滚补偿，使轮腿机构适应局部地形。

3.4 机械结构改造

整车在博思威龙 v 车模基础上，增加前方 tof 安装打印件、后侧无刷驱动安装板。主体电路板使用一块碳板固定，并在上方碳板架设 gps 与摄像头，布局考虑整体重心分布，并且使用打印件对摄像头镜头进行保护防止撞伤。电池安装在车体下方，使用专门的电池盒放置，防止电池晃动或遭受猛烈撞击。Tof 安装件与无刷驱动安装板均利用 L 型支架上预留的螺孔固定。

轮胎采用的是 90 mm 越野胎，排水抓地性能良好。为了适配胎皮与电机，同时维持轮腿跳跃能力、缓冲能力、抓地性和惯导准确性自行设计了较硬但有韧性的 tpu 轮胎填充，及适配胎皮大小与运动强度的轮毂。

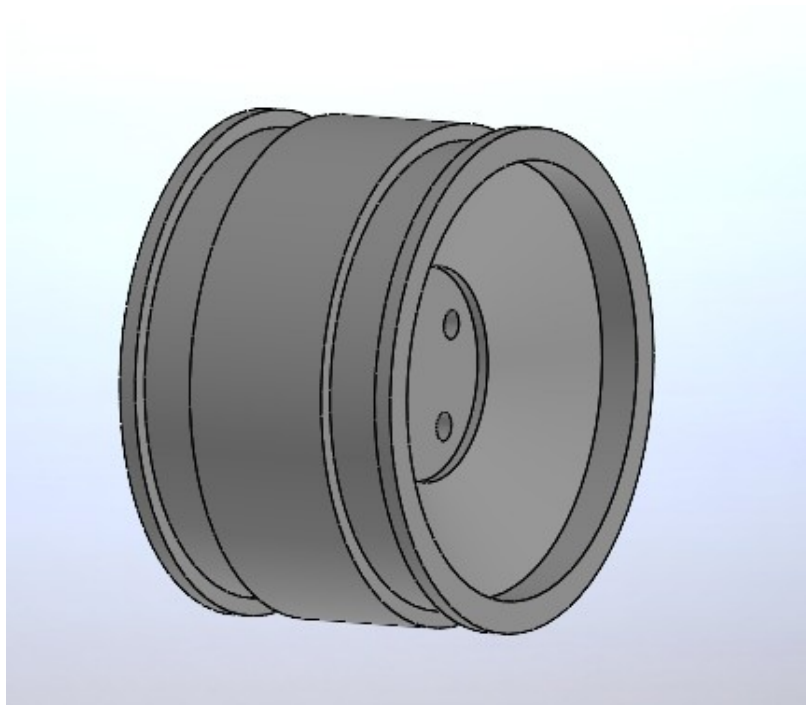


图 3.3 轮毂设计图

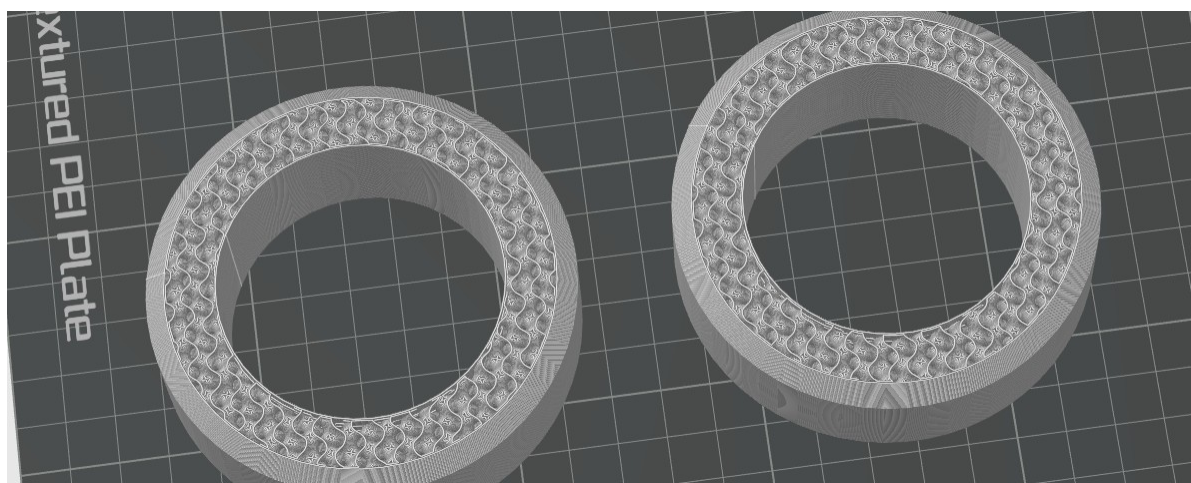


图 3.4 轮胎填充切片图

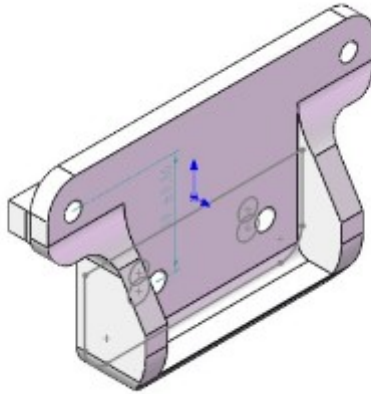


图 3.5 TOF 安装板结构图

3.5 装配与布置原则

- 电池和功率板靠近车体中心布置，减少大电流线束长度；
- 摄像头支架保持足够高度和刚度，避免振动引起视野漂移；
- 磁编码器与磁铁保持约 2 mm 间距，并保证旋转中心对准；
- 动力线、舵机线与弱信号线分束固定，降低运动干涉和电磁耦合；
- 连接器与线束设置机械固定，避免颠簸或跳跃时松脱。

第四章 硬件系统设计

4.1 硬件模块划分

整车硬件采用主控拓展主板、CYT2BL3 双电机驱动板、四路舵机驱动板、磁编码器板及线束/信号转接模块的分层结构。竞赛配发核心 MCU 不在本文重复展开芯片内部结构，重点说明自研板卡的供电、接口、执行器驱动和板间协同。模块化设计使各板能够独立上电检查，并将高功率开关区域与传感器弱信号区域分开布置。

4.2 主控拓展主板

主控拓展主板配套 CYT4BB 核心板，集成多路稳压电源与整车外设接口。电源部分由电池母线生成系统 5 V，再生成 3.3 V 供低压数字与传感器电路使用；GPS 采用独立 5 V 支路，摄像头采用独立 3.3 V 稳压支路，以降低动力执行器动作造成的供电扰动。板载和外接资源包括 IMU、GNSS、摄像头、TFT 显示、蓝牙、按键、DL1B 测距、蜂鸣器和 SBUS 接收机。

信号接口方面，IMU 通过 SPI 接入，GNSS 和蓝牙使用 UART，DL1B 使用软件 I²C，摄像头由 CM7_1 侧采集，四路舵机控制信号以 PWM 输出，电机控制目标通过 UART1 发送至 CYT2BL3 驱动板。布局上将母线和稳压区域、数字控制区域、传感器区域和外部接口区域分区，功率走线采用较宽铜箔，弱信号线避开电机驱动和开关电源节点。

第二十一届全国大学生智能汽车竞赛技术报告

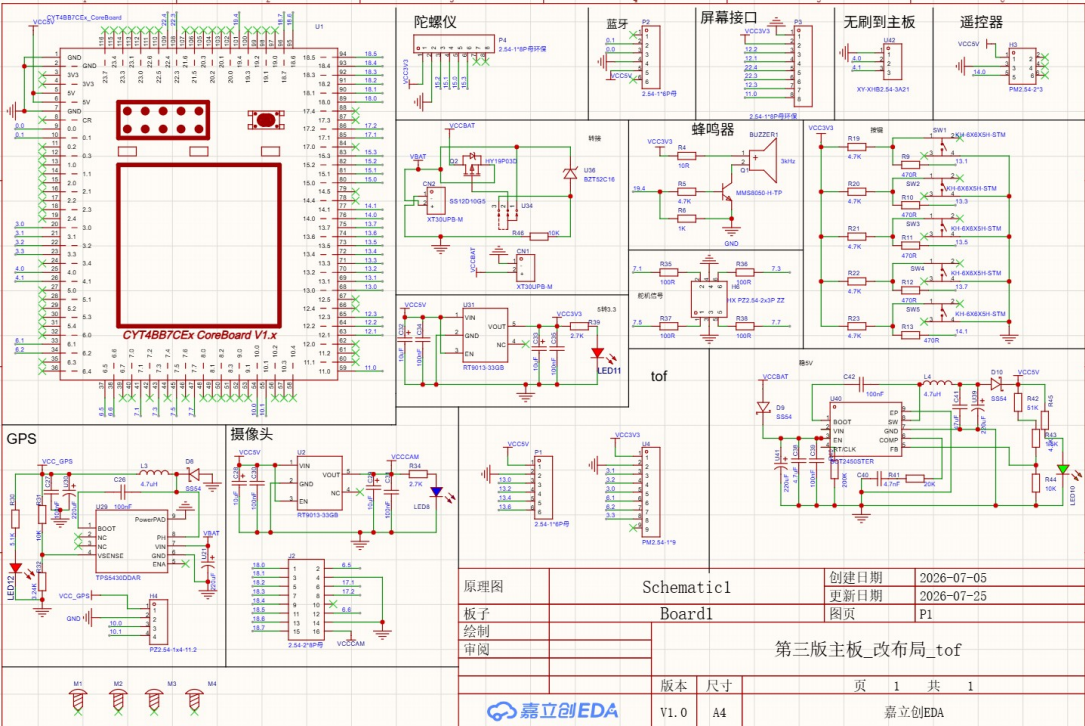


图 4.1 主控拓展主板原理图

4.3 四路舵机驱动板

舵机驱动板位于整车供电链路前端，接收 3S 电池输入并完成输入保护。四路舵机采用独立分路稳压供电，输出电压经车队确认为 7 V；各支路具有独立滤波和输出接口，信号与主控共地。主控输出 4 路 PWM 信号，各信号串联限流电阻后送至标准舵机接口。该设计可减小多个舵机同步动作造成的局部电压跌落，并便于分路排查故障。

原理图截图中的输出网络名称沿用设计文件内部命名；本报告的舵机供电数值统一采用车队确认的实际值 7 V。

四路通道并非电气隔离：系统没有隔离电源、光耦或数字隔离器，因此准确表述为“四路独立稳压、分路滤波、信号共地”。调试时应先空载确认 7 V 输出，再逐路接入舵机，最后执行四舵机同步动作并观察供电稳定性。

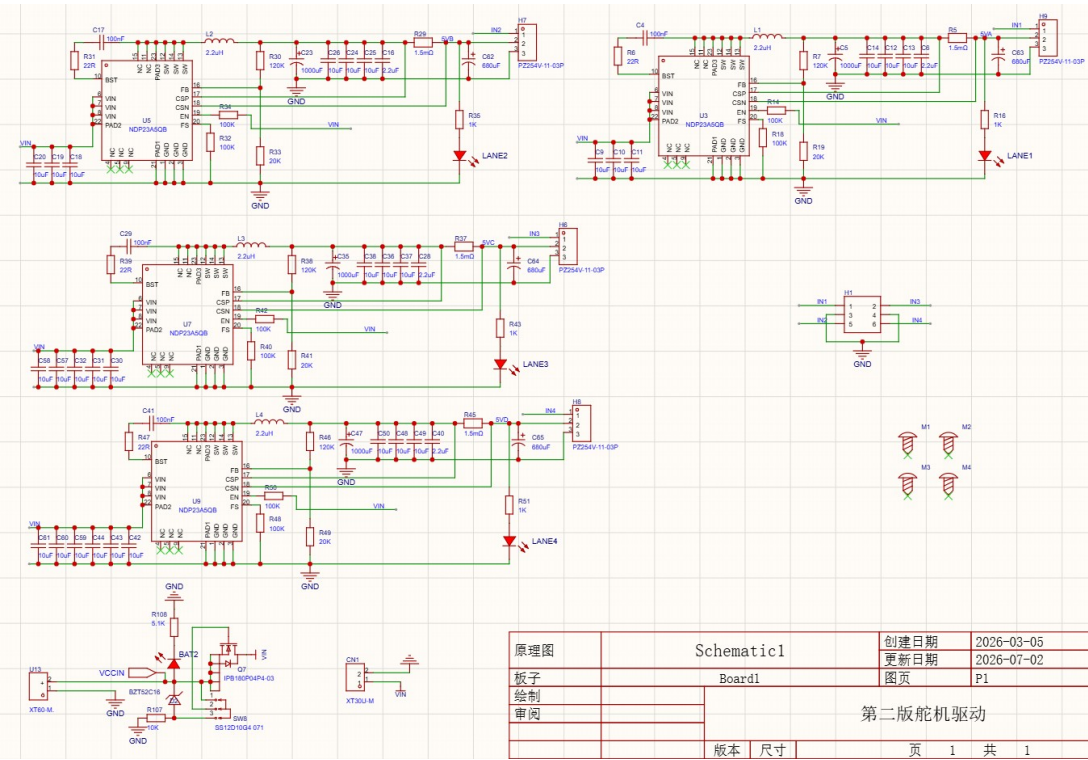


图 4.2 四路舵机驱动板原理图

4.4 CYT2BL3 双电机驱动板

双电机驱动板以 CYT2BL3 为控制核心，集成两套对称三相无刷驱动通道。板上包含低压控制电源、UART 接口、按键与指示电路、两路磁编码器 SPI 接口、预驱动器和三相 MOS 全桥。左右两路驱动电路共用系统供电和控制地，但功率回路、栅极驱动和相线输出分别布置。主控制器通过 UART1 发送左右电机占空比或速度工程量，驱动板根据磁编码器角度和程序配置生成三相 PWM。

功率区与低压控制区在布局上分区，大电流回路采用宽铜箔和多过孔，编码器与通信线远离三相开关节点。程序配置包含 800 KV、正反向牵引角 90°、最大超前角 70°、编码器量程 16383，并在启动时执行极对数标定；异常时回退为 7 极对。上述数值均属于当前程序配置，不作为电机制造商额定参数。

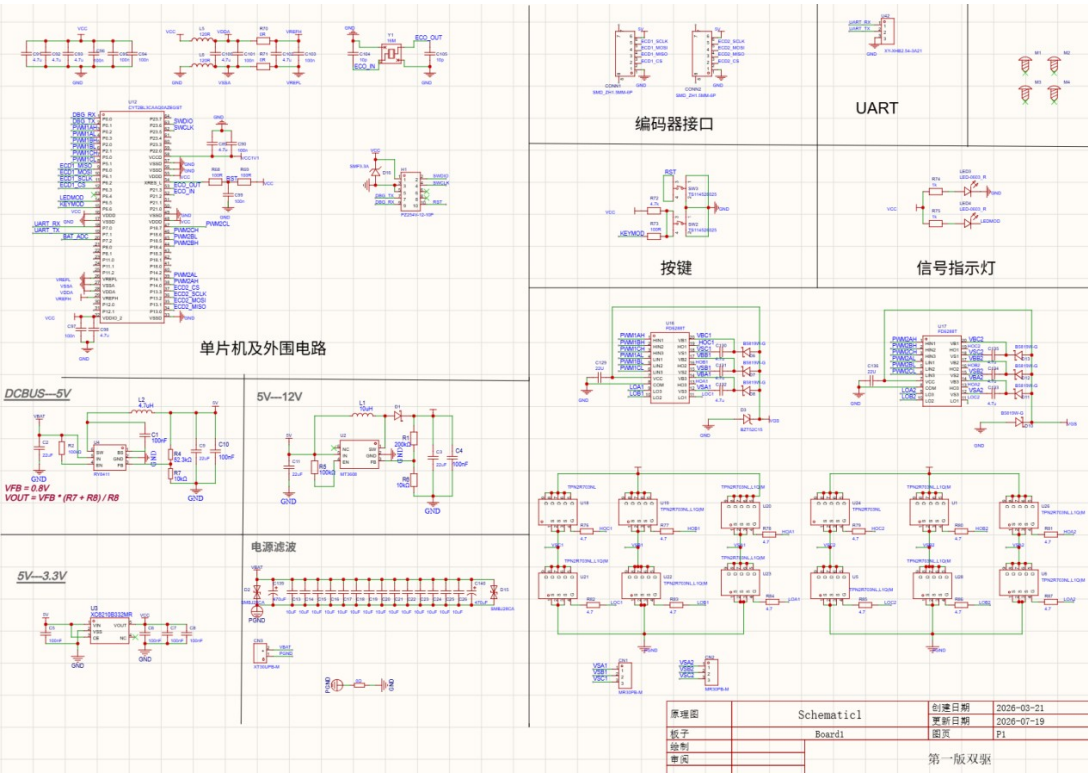


图 4.3 CYT2BL3 双路无刷电机驱动板原理图

4.5 磁编码器板与信号转接

两块 MT6816 磁编码器板安装在电机磁铁附近，由驱动板供电。编码器通过两路硬件 SPI 向 CYT2BL3 提供绝对角度，SPI 时钟为 20 MHz；驱动程序根据连续采样角度差计算轮速，用于转子同步、速度估计和主控制器反馈。编码器与磁铁间距约 2 mm，安装时需要校正同心度并检查角度连续性。

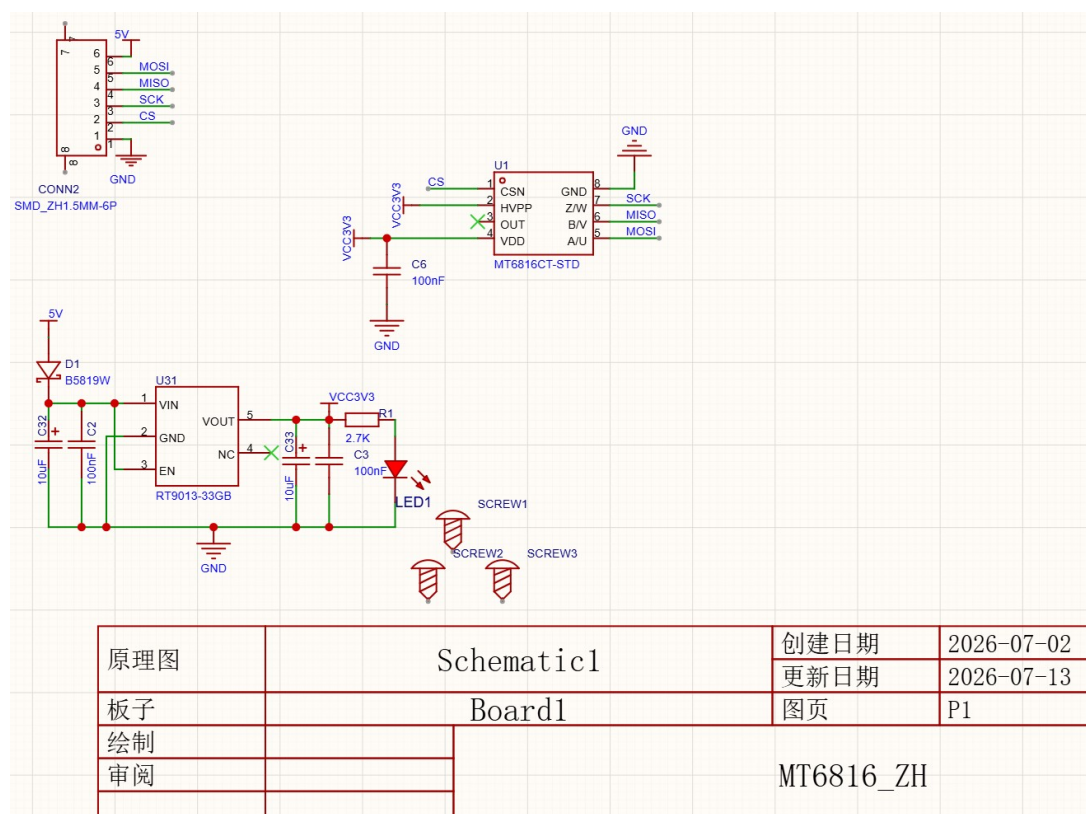


图 4.4 MT6816 磁编码器板原理图

线束及信号转接模块用于接口连接和线束整理。现有 WS2812B 灯链图与电机转接板不对应，因此本节不再展开转接板电路。

4.6 功耗与滤波电容记录

现有设计资料记录了各板功耗和板载电容容量，见表 4.1。各项峰值对应的工作状态不同，不能直接相加作为整车持续功耗。整车电容容量主要来自舵机驱动板四路降压输出的滤波电容。

表 4.1 各硬件模块功耗与电容容量记录

模块	功耗记录	电容容量
主控拓展主板	待机 0.28 W；外设全开启满载 1.15 W	972.50 μF
舵机驱动板	空载 0.06 W；四舵机同步峰值 1.8 W	6929.20 μF
双电机驱动板	空载 0.12 W；双电机满负载峰值 26 W	1139.50 μF
磁编码器板	恒定 0.13 W	20.20 μF
信号转接板	无源连接部分不计静态功耗	0 μF
合计	峰值工况不直接相加	9061.40 μF

$$P = UI$$

公式 2

功率记录应对应明确测量位置和工作状态；本文不补造未记录的测试条件。

第五章 软件系统与通信设计

5.1 开发环境与基础工程

主控制器软件使用 IAR Embedded Workbench for Arm 9.40.1 开发，基础库为逐飞 CYT4BB 开源库 V3.6.3。工程以 CM7_0 和 CM7_1 双核项目组织，底层驱动、板级初始化、实时任务、控制算法、任务状态机和人机交互相互分层。关键参数可由 Flash 覆盖代码默认值，因此本文表列默认参数仅表示程序初始配置，实际运行值以车辆 Flash 参数为准。

5.2 双核任务划分

CYT4BB双核软件分工与数据流

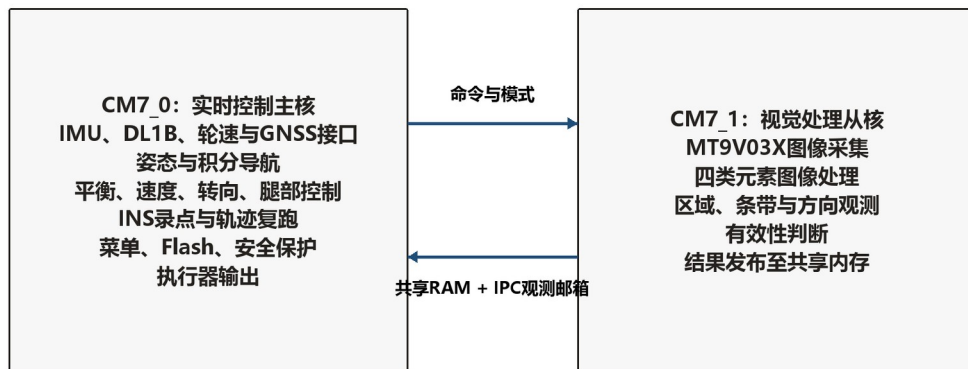


图 5.1 CYT4BB 双核软件分工与数据流

CM7_0 负责 IMU、DL1B、轮速以及导航数据采集，完成姿态更新、平衡控制、速度控制、转向控制、腿部控制、INS 录点、轨迹复跑、菜单、Flash 参数、安全保护和执行器输出。CM7_1 负责 MT9V03X 图像采集以及单边桥、颠簸、草地和坡道图像处理。图像结果写入固定布局的共享 RAM，再由 IPC 通知主核。图像处理与 1 ms 级闭环分开运行，单帧算法耗时不会直接占用主核控制中断。

5.3 初始化流程与实时调度

CM7_0 上电后依次完成系统时钟、双核通信、TFT、按键、Flash 和菜单初始化，再初始化 PID、舵机和电机驱动，随后启动 IMU、GNSS、DL1B 和 SBUS 等外设，建立 1 ms、10 ms 和 100 ms 级任务。主循环持续执行 menu_process()，高频控制与导航逻辑由 PIT 定时中断触发。

表 5.1 CM7_0 实时任务分配

通道	周期	主要任务
PIT CH0	1 ms	IMU 采集、按键扫描、调试标志
PIT CH1	1 ms	平衡、速度与腿部控制调度
PIT CH2	1 ms	旋转和转向串级控制
PIT CH10	1 ms	INS、轨迹复跑、菜单时基和元素状态
PIT CH11	1 ms	导航融合预测
PIT CH12	100 ms	产生 GNSS 10 Hz 处理标志

5.4 外设通信接口

表 5.2 主要通信接口

通信对象	接口	参数与协议
CYT4BB 与 CYT2BL3	UART1	460800 bit/s, 7 字节控制帧
SBUS 接收机	UART4	100000 bit/s, 25 字节帧
TAU1201	UART2	115200 bit/s, 10 Hz, 解析 RMC/GGA
IMU660RC	SPI2	10 MHz
DL1B	软件 I ² C	P13_0 / P13_2
双磁编码器	两路硬件 SPI	20 MHz, 绝对角度读取
CM7_0 与 CM7_1	共享 RAM + IPC	模式命令、配置序号、帧序号和分模式结果
PC 调试	UART0	JustFloat; 部分轨迹传输 4 Mbit/s

第六章 运动控制与轨迹复跑

6.1 双轮平衡串级控制

基础的双轮平衡例程通常用 IMU 倾角和一个 PID 直接生成电机 PWM^[8-9]。本车还要同时维持速度、完成元素入口对正并驱动四个腿部舵机，因此控制程序把任务拆成速度、直立角度和俯仰角速度三层。速度环给出目标俯仰角，角度环再给出目标俯仰角速度，最内层用 IMU 角速度形成平衡控制量。外层变化较慢，负责车速与稳态；内层按 1 ms 节拍工作，用来压住落地、颠簸和腿部动作带来的快速扰动。

双轮平衡串级控制结构

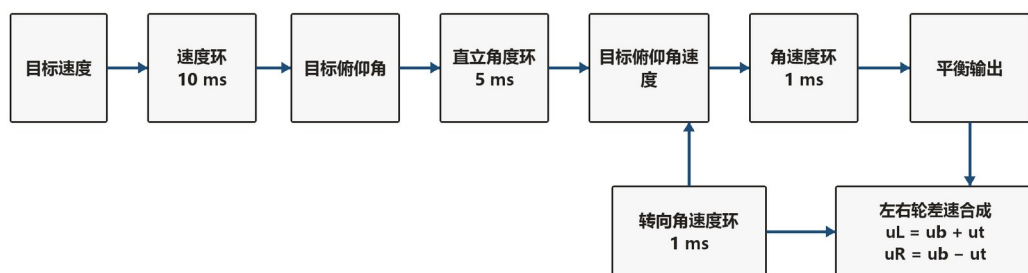


图 6.1 双轮平衡串级控制结构

转向并不是把图像误差直接写到左右电机。INS 曲率、入口曲线、单边桥、坡道、颠簸辅助和台阶动作都先换算为目标偏航角速度，主核按当前元素状态选择有效来源，再交给转向角速度环。视觉刚开始有效时采用连续混合，失效时退回记录航向或元素固定策略，避免转向目标在两帧之间突然跳变。平衡量与转向量最后按差速方式合成：

$$uL = ubalance + uturn \quad \text{公式 3}$$

$$uR = ubalance - uturn \quad \text{公式 4}$$

负号方向由工程坐标约定决定。

代码中的负速度目标只是前进方向的工程约定，不能直接当作 m/s。速度、转向和腿部状态切换时，各环节会清理或收敛上一状态留下的积分和目标值。这样处理的目的很实际：车辆从 INS 循迹切到视觉对正、从坡道对正切到航向保持，或者从跳跃落地回到常规平衡时，不把上一阶段的控制记忆带进下一阶段。

6.2 INS 录点、轨迹复跑与动态前视

6.2.1 轨迹记录与元素标记

当前 P3 分支以轮速和姿态积分得到平面坐标。普通路段按 0.2 m 空间间隔补点，车辆跨过一个间隔时，程序在前后位置之间插值，不把调度周期造成的点距抖动原样写入轨迹。复杂地形不只记录一个坐标点：每个元素保存入口点出口后的恢复点、元素类型、入口航向、恢复航向和入口至恢复点的长度。旋转、掉头与元素入口都作为事件点保留，后续复跑据此决定何时停止普通前视何时交给专用状态机。

6.2.2 Flash 保存与上位机修改

Flash 轨迹区带有魔数、版本、项目编号、坐标模式和轨迹格式。读取时先核对点数、状态值和元素元数据，写入后再回读检查。上位机可以修改轨迹点但传输同时携带点数据与元素元数据的 CRC；修改结束后，程序重新整理入口和出口对应关系，再写入 Flash。回读不一致时恢复修改前的基线数据，避免一段残缺轨迹进入正式复跑。

6.2.3 当前线段选择与脱线重挂

复跑任务每 1 ms 更新一次。程序先把当前位置投影到正在跟踪的线段，并优先沿当前线段前进，而不是每周期在整条轨迹上寻找最近点。只有横向误差超过约 0.25 m，而且局部候选线段至少改善约 0.04 m 时，才允许重新挂接。搜索范围还会被已经完成的旋转、出口点和前方事件限制，防止车辆在回环、掉头或相邻元素附近误跳到另一段轨迹。

6.2.4 动态前视与车体系曲率

前视距离随实际前进速度增大，随横向误差增大而缩短。当前分支的基础值为 0.4 m，限制在 0.3~0.6 m 之间：

$$Ld = \text{clamp}(0.4 + 0.03|v| - 0.1|ect|, 0.3, 0.6) \quad \text{公式 5}$$

v 为前进速度， ect 为当前线段横向误差。

目标点可以跨越多个普通线段，但不会越过下一个旋转、掉头或元素入口。目标点转换到车体系后，以前向坐标 x_b 和横向坐标 y_b 计算 Pure Pursuit 曲率^[7]：

$$\kappa = 2yb / (xb^2 + yb^2) \quad \text{公式 6}$$

$$\omega_{target} = -v\kappa \quad \text{公式 7}$$

低速时，目标偏航角速度按车速渐入。这一步输出的不是左右轮 PWM，角速度目标进入第 6.1 节的转向内环后才形成差速量，因此轨迹几何、视觉修正与底层电机输出之间仍保留一层可限幅、可复位的闭环。

6.2.5 元素入口曲线与出口重锚定

接近元素入口时，程序用当前轨迹方向和记录入口方向构造一段平滑过渡曲线，并沿曲线前视。摄像头在入口曲线开始时提前进入对应模式，视觉连续有效 3 帧后才逐步混入转向目标，混合时间为 250 ms，视觉附加角速度限制为 36°/s。入口判定使用带方向的入口平面，不只靠到某个点的半径判断，车辆从侧面靠近时不容易过早进入元素状态。

车辆处于元素内部时，普通坐标积分和普通 Pure Pursuit 按策略冻结，由单边桥、坡道、颠簸、草地或台阶控制器接管。通过元素后，位置恢复到录制时的出口后点，目标索引跳到该事件之后；多数元素只重置位置，只有策略明确要求时才改写航向。这样既保留元素内部允许打滑、跳跃的自由度，又让后续轨迹从一个确定的记录点重新开始。

表 6.1 P3 轨迹复跑的阶段处理

阶段	程序处理	目的
录制	0.2 m 插值补点，另存入口与出口事件	保持点距并保留地形语义
跟踪	当前线段优先，局部重挂	避免回环和相邻线段误跳
前视	速度增大时放远，横向误差大时缩短	兼顾平滑和回线能力
入口	过渡曲线、入口平面、视觉渐入	减少进入地形前的突转
元素内	专用控制器接管，普通积分按策略冻结	隔离打滑、冲击和跳跃
出口	恢复到记录恢复点并越过事件索引	重新接上后续轨迹

6.3 安全保护

- 倾倒保护：当姿态超过安全范围时停止或限制驱动输出；
- 离地保护：轮体离地或轮速异常时避免控制器持续积分；
- 跳跃状态保护：对起跳、腾空、落地阶段使用独立状态约束；
- 轮速异常与人工接管：轮速反馈不合理时降级处理，调试阶段可通过 SBUS 停机或切换模式。

第七章 感知、视觉与导航

7.1 传感器与图像输入

整车使用 5 类、6 个传感器器件：IMU660RC×1、MT9V03X×1、TAU1201×1、DL1B×1 和两只磁编码器。IMU 提供车体角速度与加速度，磁编码器通过 SPI 给出电机绝对角度，程序再由连续角度差计算轮速；DL1B 负责台阶距离触发。GNSS 接口保留用于室外数据采集和其他项目模式，当前 P3 复跑分支以轮速与姿态积分坐标为主。摄像头只处理单边桥、颠簸、人工草地和坡道四类元素，不再设置普通赛道视觉。

MT9V03X 的板端处理分辨率为 94×60。CM7_1 收到一帧后，先把灰度图复制到固定共享缓冲区，再按当前元素模式调用对应模块。场景对比度每隔一帧更新一次，用于调整坡道局部阈值。摄像头启停和模式切换可能涉及等待，因此请求由中断发出，真正的切换放在主循环执行，1 ms 控制中断不在此处阻塞。报告中的图像均来自这组 94×60 原始帧，排版时只做最近邻放大。

四类模块都采用固定容量数组和整数定点结果，没有在帧处理中申请动态内存。常见视觉库会直接提供 Hough 直线或线段接口^[10-11]；本车根据每种地形的几何范围限制候选数、斜率桶和模型数，板端只保留控制真正需要的左右结构线、横向误差、方向误差、置信度和有效标志。

7.2 单边桥引导区处理

单边桥入口的有效结构不是整块亮区，而是桥面两侧随距离收窄的边界。bridge_guide_line.c 先对全幅灰度直方图计算 Otsu 阈值，得到亮像素二值图，再调用固定整数边线拟合内核。远处方向取 $y=20$ 附近，近处横向位置取 $y=40$ 附近；左右模型至少要有共同支持行，近端宽度也要满足透视范围。拟合过程保存 Q10 斜率和 Q8 截距，既避免浮点拟合在双核结果结构中膨胀，也便于控制侧直接换算。

横向误差不是简单的像素差。程序在参考行求出左右线位置，以两线中心相对图像中心的偏移作为分子，再除以该行的可见桥面宽度，得到 Q12 归一化误差。桥面远近变化时，几像素偏移所代表的物理意义不同，宽度归一后，同一组控制参数可以覆盖较大的观察距离。方向误差由远近两个中心的变化量给出。任何一侧丢失、左右线无法配对或几何宽度不成立时，当前结果直接标为无效。

$$el = (xref - xc) / (xright - xleft)$$

公式 8

xref 为图像中心，xc 为双边线中心。

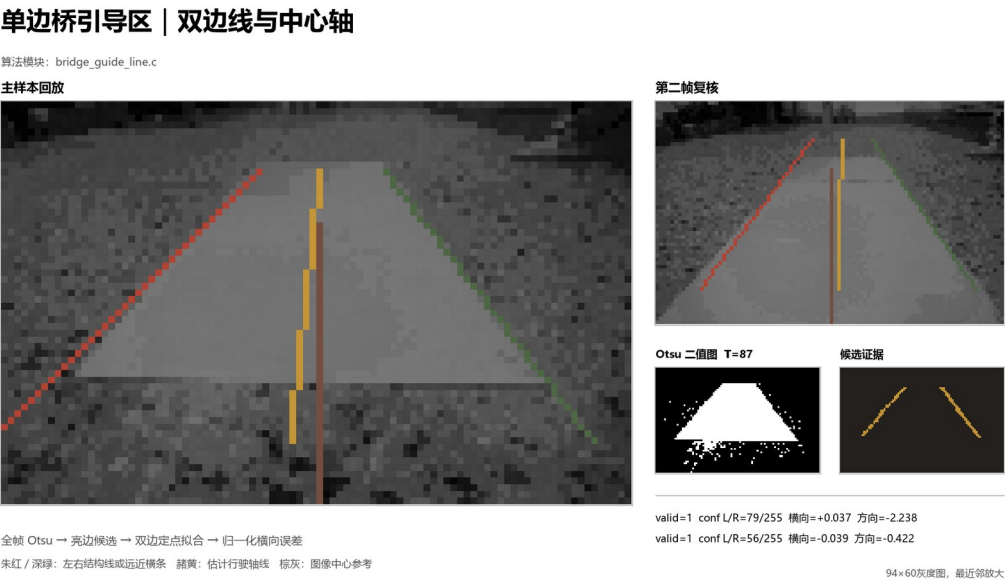


图 7.1 单边桥引导区的当前 C 算法回放

进入单边桥引导状态后，视觉先连续确认，再以有限角速度修正入口方向。若双边线中途丢失，控制器在第一次失效时保存入口参考航向，随后保持该航向通过，不在碎石噪声中反复切回视觉。车体横滚和腿高仍由姿态控制独立处理。

7.3 颠簸路段处理

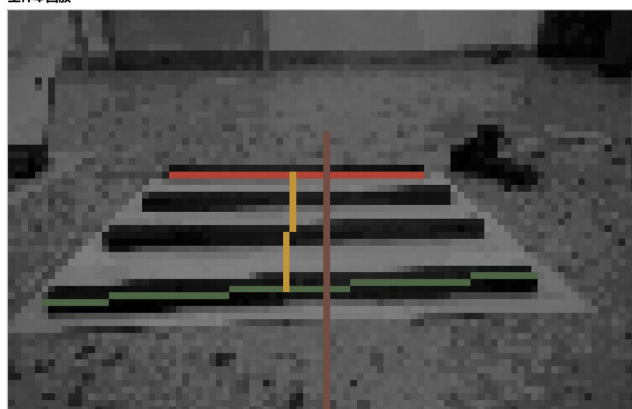
颠簸路段的横杆数量和可见长度会随距离变化，程序不假设画面里固定出现几根。bumpy_road.c 在 $x=4\sim 89$ 、 $y=14\sim 51$ 范围内工作，用全帧较暗 15% 像素确定阈值。每一行寻找连续暗像素游程，允许中间夹有不超过 2 个亮像素，长度不足 6 像素的游程不再进入下一步。相邻行游程有足够重叠时合并为组件，组件数量限制为 24，随后按支持点、跨行范围、残差和斜率拟合成最多 12 条横杆模型。

保留下来的横杆按远到近排序。动态规划寻找一条长度可变的序列，前后横杆的纵向间隔、斜率差、中心横移和透视尺度必须连续；一般至少需要 3 根，只有证据较强时才接受 2 根。最终输出最远和最近横杆端点、序列数量、置信度，以及由两条中心连线得到的道路轴线。横向误差在参考点 $x=47$ 、 $y=38$ 处按插值宽度归一，因此它表达的是车辆在横杆通道中的相对位置，而不是某一根横杆的局部缺口。

颠簸路段 | 横条序列与道路轴线

算法模块: bumpy_road.c

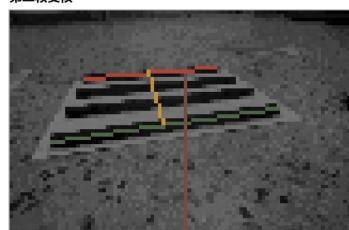
主样本回放



逐行暗像素游程 → 跨行组件 → 横条拟合 → 可变长度序列

朱红 / 深绿: 左右结构线或远近横条 蓝黄: 估计行驶轴线 棕灰: 图像中心参考

第二帧复核



纵向明暗响应 暗阈值 $T=31$



候选证据



valid=1 bars=4 conf=202 横向=+0.080 轴线=-0.059

valid=1 bars=4 conf=207 横向=+0.045 轴线=+0.285

94×60灰度图，最近邻放大

图 7.2 颠簸路段的当前 C 算法回放

板端控制会按帧序号保持最近一次有效方向，并给视觉输出设置限幅。真正的颠簸腿部动作仍由元素状态、姿态冲击和机械条件共同决定；视觉负责入口和轴线辅助，不单独触发大幅腿部动作。若横条序列消失，转向退回入口航向保持。

7.4 坡道处理

坡道模块不先分割整块坡面，而是直接寻找两侧由暗背景向亮坡面过渡的边线。ramp_side_line.c 对每个候选位置计算 3 像素局部均值，比较边线内外亮度、有符号梯度和局部亮度抬升。阈值会跟随场景对比度调整。每行最多保留 3 个候选，斜率空间量化为 32 个桶，每侧只留下评分最高的 4 个模型，避免无界枚举。

单侧模型至少覆盖 8 行、跨越 6 行，最大断行不超过 3 行，定点残差受限。左右模型配对时还要满足近端足够宽、两侧向下展开、支持行相对均衡，并使消失点落在预定区域。通过配对后，程序在 $y=30$ 和 $y=55$ 附近比较双边中心，分别生成方向误差和接近端宽度归一的横向误差。调试图只显示通过局部判据的候选点，不把整个亮坡面涂成一个连通块。

坡道入口 | 侧边线与消失点约束

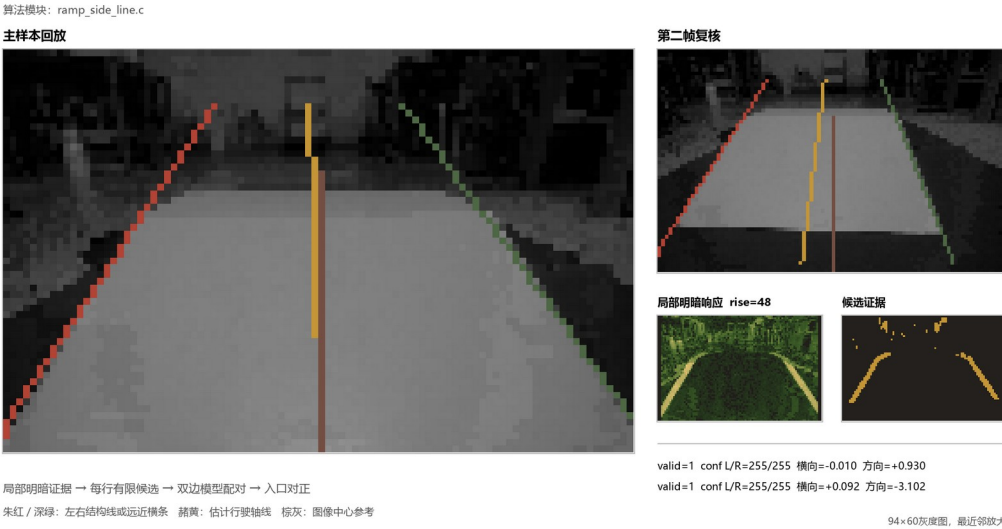


图 7.3 坡道入口的当前 C 算法回放

主核的坡道对正不是一个长期视觉 PID，而是 VISUAL、READY、YAW_HOLD 和 FALLBACK 几种状态。视觉目标先低通，再限制单帧变化；左右误差稳定、车体偏航和陀螺角速度进入允许范围后，程序锁定记录的入口航向，坡内改为航向保持。视觉在对正阶段丢失时也直接退到入口航向，不让亮斑重新夺回转向。

7.5 人工草地处理

草地样本的内部纹理不稳定，但左右边缘在多数帧仍能形成一暗一亮的有符号梯度。grass_road.c 在 $x=2\sim 91$ 、 $y=7\sim 55$ 区域统计较暗 15% 像素作为阈值，从 4 个种子行分别尝试左右边缘。左边要求外侧亮、内侧暗，右边符号相反；候选只在上一行附近的有限窗口内向前跟踪，随后用 Q10 斜率和 Q8 截距重新拟合。左右线必须分别向左、向右展开，共同覆盖至少 7 行，远近宽度和透视展开量也要成立。

种子跟踪失败时，程序进入有界回退：每行最多 5 个候选，斜率划分 18 个桶，每侧保留 4 个模型，再按边线对比度和双边几何配对。若左边缘仍明显退化，还会在 13×13 局部高通图上做一次 Otsu 恢复。回退仍使用固定数组，不会因为纹理增多而增长内存。输出为双侧置信度、平均方向斜率和归一化横向误差。

人工草地 | 双侧边界跟踪

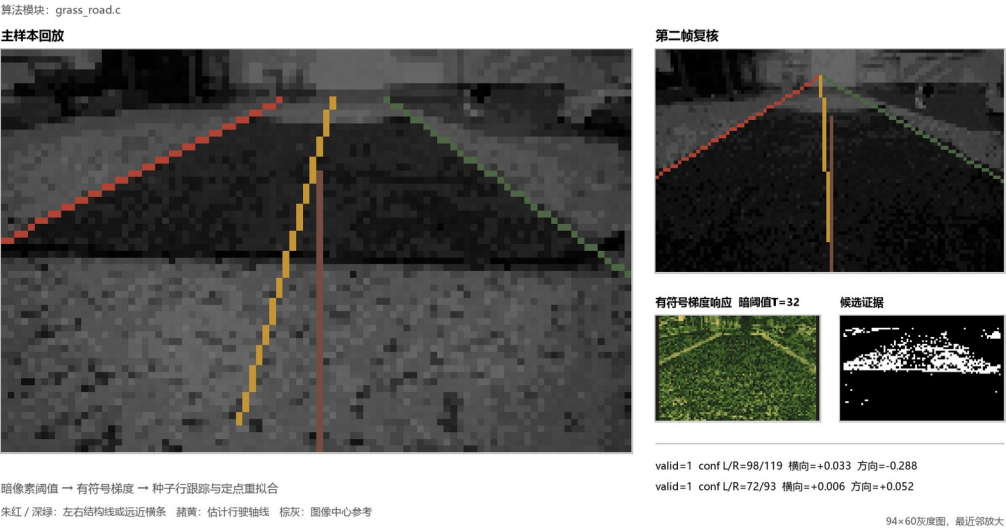


图 7.4 人工草地的当前 C 算法回放

草地视觉模块目前用于离线回放和参数调试，P3 默认配置关闭草地摄像头转向。正式流程进入草地后保持记录航向和元素速度目标，因此报告将草地模块列为实验性模式，与比赛流程分开记录。

7.6 观测发布与导航配合

共享结果区采用固定布局。CM7_1 写入结果前后更新序号，并附带配置序号、帧序号和当前图像模式；完成缓存清理后再发 IPC 通知。CM7_0 读取前使本核缓存失效，只有两次读到的序号一致，且模式、配置和帧号都匹配时才接受结果。各模式的获取函数还检查数据是否超时。这样可以避免主核读到一半新一半旧的结构，也不会把上一元素的最后一帧沿用到下一元素。

方向误差和横向误差进入统一融合器。横向误差先经过死区，偏差较小时以方向误差为主，车辆已经明显偏到一侧时逐渐提高横向误差权重，最终再乘增益并限幅：

$$u_{vision} = K[wl \cdot el + (1 - wl) \cdot eh]$$

公式 9

wl 随|el|增大；el 和 eh 均先按各元素尺度换算。

融合结果仍只是候选目标。入口曲线、台阶动作、坡道对正、颠簸辅助、单边桥航向保持和 INS 曲率都有明确优先级。转向来源改变时，主核采用确认帧、目标保持、渐入渐出或 PID 复位，使控制链知道这次变化来自状态切换，而不是把它当成突然出现的道路曲率。

表 7.1 元素观测、控制接入与失效处理

元素	板端观测	P3 当前使用方式	结果失效后
单边桥	双边线、横向和方向误差	视觉引导开启，入口渐入	锁定入口航向保持
颠簸	横条序列、道路轴线、置信度	视觉方向与冲击辅助开启	保持入口航向，姿态状态继续
坡道	双侧结构线、横向和方向误差	入口对正开启，稳定后锁航向	退回入口航向
人工草地	双侧边界、横向和方向误差	模块已实现，P3 默认关闭	按 INS 记录航向和固定速度
台阶	视觉模块保留但默认不参与	DL1B 连续距离触发跳跃	不依赖视觉回退

本章四幅处理图由报告生成脚本直接调用当前工程中的 bridge_guide_line.c、bumpy_road.c、ramp_side_line.c 和 grass_road.c 得到，不是按说明文字重新绘制的示意结果。图内数值为对应帧的定点结果换算值，只用于核对处理链；由于素材没有统一人工标签，它们不能代替整车识别率或通过率统计。

第八章 整车调试与元素验证

8.1 离线回放检查

四个文件夹共有 64 帧 94×60 灰度图。回放程序把工程中的 bridge_guide_line.c、bumpy_road.c、ramp_side_line.c 和 grass_road.c 编译为主机动态库，逐帧调用与板端相同的入口函数和默认参数。第 7 章的图不是人工勾边：双边线斜率、横向误差、横条数量和置信度都来自 C 结构体返回值。每组挑选两帧排版，其余帧仍用于检查有效标志、左右置信度和输出方向是否连续。

中间图按算法实际需要选择。单边桥显示 Otsu 二值图和边线候选；坡道显示局部明暗响应和有限候选；颠簸显示暗像素阈值与横条序列；草地显示有符号梯度、暗像素阈值和双侧拟合。它们用于定位阈值、候选或配对阶段的问题不再用一套通用的归一化、形态学和连通域界面解释所有元素。

离线回放只能说明代码在现有图像上给出了什么观测，不能代替整车通过试验。当前素材没有逐帧人工标签，因此本报告不把有效标志统计成识别率，也不以视觉图片推算整车通过率。

8.2 板端接口与时序

CM7_1 完成当前模式后，先写固定 16 字节的分模式结果，再更新帧序号与发布序号，清理共享区缓存后发送 IPC 通知。CM7_0 读取时先使缓存失效，并检查发布序号前后一致、配置序号一致、图像模式一致且结果没有超时。模式切换由主循环执行，避免摄像头启停过程占住 1 ms 中断。

调试串口应同步记录帧号、配置序号、模式、有效标志、左右置信度、横向误差、方向误差、IMU 姿态和最后采用的转向来源。这样可以区分算法没有形成有效模型、共享结果没有更新、结果被状态机拒绝以及主核主动限幅四种情况。

8.3 单边桥与坡道联调

单边桥和坡道都输出左右线，但进入控制的方式不同。单边桥先核对归一化横向误差的符号，再检查视觉连续确认、250 ms 渐入和丢线后航向锁定。坡道先核对左右线配对、近端宽度和方向误差，再观察 VISUAL、READY、YAW_HOLD 状态是否按顺序推进；航向锁定后，视觉不再持续拉扯坡内方向。

表 8.1 单边桥与坡道联调重点

项目	单边桥	坡道
主要观测	Otsu 亮边、双边中心和宽度	局部明暗侧边线、消失点约束
先核对	偏差符号、确认帧、渐入过程	模型配对、稳定帧、航向锁定
组合信息	入口航向、车体横滚	入口航向、偏航角速度、车体俯仰
失效处理	锁定入口航向继续	直接退回入口航向

8.4 颠簸与人工草地联调

颠簸路段先看暗游程能否合并成连续横杆，再看动态规划得到的横条数量、远近端点和道路轴线是否随距离平滑变化。视觉方向按帧保持并限幅，腿高和姿态动作仍由冲击与元素状态安排。若碎石纹理只形成孤立短线，模型支持数跨行范围和序列连续性应把它挡在有效结果之外。

人工草地先分别检查左右有符号梯度，再看种子跟踪和回退模型使用了哪条路径。草丝反光或阴影会让一侧变弱，因此左右置信度要分开记录。P3 默认关闭草地视觉转向，调试结果只写入观测日志；正式状态保持积分轨迹记录的航向和元素速度目标。

附录 A 主要参数

表 A-1 整车主要参数

项目	参数	数值或配置
整车	外形尺寸	213 mm × 185 mm × 270 mm
整车	质量	1.35 kg，含电池
车轮	直径 / 轮距	90 mm / 175 mm
轮腿	物理连杆长度	大腿 70 mm，小腿 90 mm
轮腿	基础腿高 / 范围	45 mm / 35~100 mm
电池	规格	3S 1800 mAh 45C
舵机	供电	7 V，四路独立稳压
图像	处理分辨率	94×60
电机程序	KV / 减速比	800 KV / 19.2
编码器	SPI 时钟	20 MHz
主控与驱动	UART	460800 bit/s，7 字节帧

附录 B 原理图索引

表 B-1 核心原理图索引

图号	原理图	章节
图 4.1	主控拓展主板原理图	4.2
图 4.2	四路 7 V 舵机驱动板原理图	4.3
图 4.3	CYT2BL3 双路无刷电机驱动板原理图	4.4
图 4.4	MT6816 磁编码器板原理图	4.5

附录 C 核心算法程序

本附录代码直接取自本车当前工程。为控制篇幅，保留双轮控制、各地形识别入口和视觉误差融合的核心程序，平台初始化、底层驱动及重复的静态辅助函数不再展开。

C.1 双轮控制 PID 程序

文件：project/code/control/pid.c

```

#include "zf_common_headfile.h"
#include "pid.h"
#include "pid_user_config.h"
volatile int pwm_out = 0;
pid_3loop_t g_pid;
float pid_pos_update_dt(pid_param *pid, float feedback, float dt_ms)
{
    float dt_s;
    float error;
    float p_term;
    float d_term;
    float new_integral;
    float output_unclamped;
    float output;

    if (0 == pid)
    {
        return 0.0f;
    }

    dt_s = dt_ms * 0.001f;
    if (dt_s <= 0.0f)
    {
        dt_s = 1.0e-6f;
    }

    error = pid->target - feedback;

    // P
    p_term = pid->kp * error;
    // D
    d_term = pid->kd * ((error - pid->last_error) / dt_s);

    float delta_integral = error * dt_s;

    new_integral = pid->integral + delta_integral;
    new_integral = pid_clamp_f32(new_integral, pid->integral_min, pid->integral_max);
    pid->integral = new_integral;
    output_unclamped = p_term + pid->ki * pid->integral + d_term;
    output = pid_clamp_f32(output_unclamped, pid->out_min, pid->out_max);

    pid->last_error = error;
    pid->last_feedback = feedback;
    pid->output = output;
    return output;
}

float pid_add_update_dt_i_scale(pid_param *pid,
                                float feedback,
                                float dt_ms,
                                float integral_scale)
{
    float dt_s;
    float error;
    float p_delta;
    float i_delta;
    float d_delta;
    float output_candidate;

    if (0 == pid)

```

```

{
    return 0.0f;
}

dt_s = dt_ms * 0.001f;
if (dt_s <= 0.0f)
{
    dt_s = 1.0e-6f;
}

error = pid->target - feedback;

p_delta = pid->kp * (error - pid->last_error);
integral_scale = pid_clamp_f32(integral_scale, 0.0f, 1.0f);
i_delta = pid->ki * error * dt_s * integral_scale;
d_delta = pid->kd * ((error - 2.0f * pid->last_error + pid->last_last_error) / dt_s);

// Anti-windup: when output is already saturated and the current error
// would keep pushing in the same direction, stop accumulating I increment.
if (((pid->output >= pid->out_max) && (error > 0.0f)) ||
    ((pid->output <= pid->out_min) && (error < 0.0f)))
{
    i_delta = 0.0f;
}

output_candidate = pid->output + p_delta + i_delta + d_delta;

pid->last_last_error = pid->last_error;
pid->last_error = error;

pid->output = pid_clamp_f32(output_candidate, pid->out_min, pid->out_max);
return pid->output;
}

float pid_add_update_dt(pid_param *pid, float feedback, float dt_ms)
{
    return pid_add_update_dt_i_scale(pid, feedback, dt_ms, 1.0f);
}

void pid_init(pid_param *pid, float kp, float ki, float kd, float out_min, float out_max)
{
    if (0 == pid)
    {
        return;
    }

    pid->kp = kp;
    pid->ki = ki;
    pid->kd = kd;

    pid->out_min = out_min;
    pid->out_max = out_max;

    pid->integral_min = out_min * 2.0f;
    pid->integral_max = out_max * 2.0f;

    pid->target = 0.0f;
    pid->integral = 0.0f;
    pid->last_error = 0.0f;
    pid->last_last_error = 0.0f;
}

```

```

    pid->last_feedback = 0.0f;
    pid->output = 0.0f;
}

void pid_start_init(void)
{
    pid_init(&g_pid.vel, PID_VEL_KP, PID_VEL_KI, PID_VEL_KD, PID_VEL_OUT_MIN, PID_VEL_OUT_MAX);
    pid_init(&g_pid.ang, PID_ANG_KP, PID_ANG_KI, PID_ANG_KD, PID_ANG_OUT_MIN, PID_ANG_OUT_MAX);
    pid_init(&g_pid.angvel, PID_ANGVEL_KP, PID_ANGVEL_KI, PID_ANGVEL_KD, PID_ANGVEL_OUT_MIN,
PID_ANGVEL_OUT_MAX);
    pid_init(&g_pid.turn, PID_TURN_KP, PID_TURN_KI, PID_TURN_KD, PID_TURN_OUT_MIN,
PID_TURN_OUT_MAX);
    pid_init(&g_pid.height, PID_ROLL_HEIGHT_KP, PID_ROLL_HEIGHT_KI, PID_ROLL_HEIGHT_KD,
PID_ROLL_HEIGHT_OUT_MIN, PID_ROLL_HEIGHT_OUT_MAX);

}

```

C.2 单边桥引导区识别程序

文件：project/code/vision/bridge_guide_line.c

```

#include "bridge_guide_line.h"

#include <string.h>

#define BRIDGE_GUIDE_PIXELS (RAMP_WHITE_LINE_WIDTH *
RAMP_WHITE_LINE_HEIGHT)

static uint8 bridge_guide_binary[BRIDGE_GUIDE_PIXELS];
static uint16 bridge_guide_histogram[256];

static uint8 bridge_guide_otsu_threshold(void)
{
    uint32 total_sum = 0u;
    uint32 foreground_sum = 0u;
    uint16 foreground_count = 0u;
    uint64 best_score = 0u;
    uint8 best_threshold = 0u;
    uint16 threshold;

    for (threshold = 0u; threshold < 256u; threshold++)
    {
        total_sum += (uint32)threshold * bridge_guide_histogram[threshold];
    }
    for (threshold = 0u; threshold < 255u; threshold++)
    {
        uint16 background_count;
        int64 difference;
        uint64 score;

        foreground_count = (uint16)(foreground_count +
bridge_guide_histogram[threshold]);
        if (foreground_count == 0u) continue;
        background_count = (uint16)(BRIDGE_GUIDE_PIXELS - foreground_count);
        if (background_count == 0u) break;
        foreground_sum += (uint32)threshold * bridge_guide_histogram[threshold];
        difference = (int64)foreground_sum * BRIDGE_GUIDE_PIXELS
            - (int64)total_sum * foreground_count;
        score = (uint64)(difference * difference)
            / ((uint32)foreground_count * background_count);
        if (score > best_score)
        {
            best_score = score;
            best_threshold = (uint8)threshold;
        }
    }
    return best_threshold;
}

static void bridge_guide_binarize_otsu(const uint8 *gray)
{
    uint16 index;
    uint8 threshold;

    memset(bridge_guide_histogram, 0, sizeof(bridge_guide_histogram));
    for (index = 0u; index < BRIDGE_GUIDE_PIXELS; index++)
    {
        bridge_guide_histogram[gray[index]]++;
    }
    threshold = bridge_guide_otsu_threshold();
    for (index = 0u; index < BRIDGE_GUIDE_PIXELS; index++)

```

```

    {
        bridge_guide_binary[index] = (gray[index] > threshold) ? 255u : 0u;
    }
}

uint8 bridge_guide_line_detect(
    const uint8 *gray,
    bridge_guide_line_result_t *result,
    int16 left_boundary[RAMP_WHITE_LINE_HEIGHT],
    int16 right_boundary[RAMP_WHITE_LINE_HEIGHT],
    uint8 *debug_edge)
{
    static const ramp_white_line_params_t params =
        {12u, 5u, 65u, 121u, 410u, 3u, 4u, 3u, 65u, 0u, 12u};
    static const ramp_white_line_profile_t profile =
        {0, 0, 0, 0, 0, 0, 2048u, 6u, 0u, 1u, 0u, 0u, 0u,
        BRIDGE_GUIDE_HEADING_FAR_Y,
        BRIDGE_GUIDE_HEADING_NEAR_Y,
        BRIDGE_GUIDE_LATERAL_Y};

    if ((gray == NULL) || (result == NULL)) return 0u;
    bridge_guide_binarize_otsu(gray);
    return ramp_white_line_detect_profile(bridge_guide_binary, &params,
        &profile, result,
        left_boundary, right_boundary,
        debug_edge);
}

uint32 bridge_guide_line_workspace_bytes(void)
{
    return (uint32)(sizeof(bridge_guide_binary)
        + sizeof(bridge_guide_histogram)
        + ramp_white_line_workspace_bytes());
}

```

C.3 颠簸路段识别入口

文件：project/code/vision/bumpy_road.c

```

uint8 bumpy_road_detect(const uint8 *gray,
                        const bumpy_road_params_t *params_in,
                        bumpy_road_result_t *result)
{
    bumpy_road_params_t params;
    bumpy_road_component_t components[BUMPY_ROAD_MAX_COMPONENTS];
    bumpy_road_line_t lines[BUMPY_ROAD_MAX_LINES];
    uint8 component_count = 0u;
    uint8 line_count = 0u;
    uint8 threshold;
    uint8 y;

    if ((gray == NULL) || (params_in == NULL) || (result == NULL))
        return 0u;
    memset(result, 0, sizeof(*result));
    result->reason = BUMPY_ROAD_REASON_INVALID_ARGUMENT;
    params = *params_in;
    params.dark_percent = bumpy_road_clamp_u8(params.dark_percent, 5u, 35u);
    params.min_run_width = bumpy_road_clamp_u8(params.min_run_width, 3u, 24u);
    params.max_bright_gap = bumpy_road_clamp_u8(params.max_bright_gap, 0u, 4u);
    params.link_overlap = bumpy_road_clamp_u8(params.link_overlap, 1u, 20u);
    params.min_span = bumpy_road_clamp_u8(params.min_span, 10u, 70u);
    params.min_support = bumpy_road_clamp_u8(params.min_support, 6u, 120u);
    params.max_bar_gap = bumpy_road_clamp_u8(params.max_bar_gap, 4u, 20u);
    params.min_chainBars = bumpy_road_clamp_u8(params.min_chainBars, 2u, 6u);
    params.reference_x = bumpy_road_clamp_u8(params.reference_x, BUMPY_ROAD_X_MIN,
BUMPY_ROAD_X_MAX);
    params.reference_y = bumpy_road_clamp_u8(params.reference_y, BUMPY_ROAD_Y_MIN,
BUMPY_ROAD_Y_MAX);
    threshold = bumpy_road_dark_threshold(gray, params.dark_percent);

    for (y = BUMPY_ROAD_Y_MIN; y <= BUMPY_ROAD_Y_MAX; y++)
    {
        uint8 x;
        uint8 run_start = 0u;
        uint8 last_dark = 0u;
        uint8 bright_count = 0u;
        uint8 in_run = 0u;
        uint8 runs_this_row = 0u;

        for (x = BUMPY_ROAD_X_MIN; x <= BUMPY_ROAD_X_MAX; x++)
        {
            if (gray[(uint32)y * BUMPY_ROAD_WIDTH + x] <= threshold)
            {
                if (in_run == 0u)
                {
                    run_start = x;
                    in_run = 1u;
                }
                last_dark = x;
                bright_count = 0u;
            }
            else if (in_run != 0u)
            {
                bright_count++;
                if (bright_count > params.max_bright_gap)
                {
                    uint8 run_width = (uint8)(last_dark - run_start + 1u);
                    if ((run_width >= params.min_run_width) && (runs_this_row < 12u))
                    {

```

```

uint8 i;
uint8 primary = 0xFFu;
uint8 best_overlap = 0u;
for (i = 0u; i < component_count; i++)
{
    uint8 overlap;
    if (components[i].last_y != (uint8)(y - 1u)) continue;
    overlap = bumpy_road_overlap(run_start, last_dark,
                                components[i].last_start, components[i].last_end);
    if ((overlap >= params.link_overlap) && (overlap > best_overlap))
    {
        primary = i;
        best_overlap = overlap;
    }
}
if (primary == 0xFFu)
{
    if (component_count < BUMPY_ROAD_MAX_COMPONENTS)
    {
        bumpy_road_component_t *component = &components[component_count++];
        memset(component, 0, sizeof(*component));
        component->first_x = run_start;
        component->last_x = last_dark;
        bumpy_road_component_add(component, gray, threshold, y, run_start, last_dark);
    }
}
else
{
    i = 0u;
    while (i < component_count)
    {
        if ((i != primary) && (components[i].last_y == (uint8)(y - 1u))
            && (bumpy_road_overlap(run_start, last_dark,
                                components[i].last_start, components[i].last_end) >= params.link_overlap))
        {
            bumpy_road_component_absorb(&components[primary], &components[i]);
            bumpy_road_remove_component(components, &component_count, i);
            if (i < primary) primary--;
            continue;
        }
        i++;
    }
    bumpy_road_component_add(&components[primary], gray, threshold, y, run_start, last_dark);
}
runs_this_row++;
}
in_run = 0u;
bright_count = 0u;
}
}
if ((in_run != 0u) && ((uint8)(last_dark - run_start + 1u) >= params.min_run_width)
    && (runs_this_row < 12u))
{
    /* Feed an end-of-row run through the next scan by treating its final
     * bright gap as zero. This duplicate-free compact block mirrors the
     * normal path above. */
    uint8 i;
    uint8 primary = 0xFFu;
    uint8 best_overlap = 0u;

```

```

for (i = 0u; i < component_count; i++)
{
    uint8 overlap;
    if (components[i].last_y != (uint8)(y - 1u)) continue;
    overlap = bumpy_road_overlap(run_start, last_dark,
                                components[i].last_start, components[i].last_end);
    if ((overlap >= params.link_overlap) && (overlap > best_overlap))
    {
        primary = i;
        best_overlap = overlap;
    }
}
if (primary == 0xFFu)
{
    if (component_count < BUMPY_ROAD_MAX_COMPONENTS)
    {
        bumpy_road_component_t *component = &components[component_count++];
        memset(component, 0, sizeof(*component));
        component->first_x = run_start;
        component->last_x = last_dark;
        bumpy_road_component_add(component, gray, threshold, y, run_start, last_dark);
    }
}
else
{
    i = 0u;
    while (i < component_count)
    {
        if ((i != primary) && (components[i].last_y == (uint8)(y - 1u))
            && (bumpy_road_overlap(run_start, last_dark,
                                   components[i].last_start, components[i].last_end) >= params.link_overlap))
        {
            bumpy_road_component_absorb(&components[primary], &components[i]);
            bumpy_road_remove_component(components, &component_count, i);
            if (i < primary) primary--;
            continue;
        }
        i++;
    }
    bumpy_road_component_add(&components[primary], gray, threshold, y, run_start, last_dark);
}
}

for (y = 0u; y < component_count; y++)
{
    bumpy_road_line_t line;
    if ((bumpy_road_component_to_line(&components[y], &params, &line) != 0u)
        && (bumpy_road_line_has_local_contrast(gray, &line) != 0u))
        bumpy_road_add_line(lines, &line_count, &line);
}
if (line_count == 0u)
{
    result->reason = BUMPY_ROAD_REASON_NO_BAR;
    return 0u;
}
bumpy_road_sort_lines(lines, line_count);
if (bumpy_road_select_nearest_line(lines, line_count, &params, result) == 0u)
{
    result->reason = BUMPY_ROAD_REASON_NO_BAR;
}

```

```
    return 0u;  
}  
return 1u;  
}
```

C.4 坡道侧边线识别入口

文件：project/code/vision/ramp_white_line.c

```

uint8 ramp_white_line_detect_profile(const uint8 *gray,
                                     const ramp_white_line_params_t *params,
                                     const ramp_white_line_profile_t *profile,
                                     ramp_white_line_result_t *result,
                                     int16 left_boundary[RAMP_WHITE_LINE_HEIGHT],
                                     int16 right_boundary[RAMP_WHITE_LINE_HEIGHT],
                                     uint8 *debug_edge)
{
    ramp_model_t left_models[RAMP_TOP_MODELS];
    ramp_model_t right_models[RAMP_TOP_MODELS];
    const ramp_model_t *best_left = NULL;
    const ramp_model_t *best_right = NULL;
    int32 best_score = -2 147 483 647;
    uint8 close_surface;
    uint8 left_count = 0u;
    uint8 right_count = 0u;
    uint8 i;
    uint8 j;

    if ((gray == NULL) || (params == NULL) || (profile == NULL) || (result == NULL)) return 0u;
    memset(result, 0, sizeof(*result));
    result->reason = RAMP_WHITE_LINE_REASON_INVALID_ARGUMENT;
    if ((profile->heading_far_y >= RAMP_WHITE_LINE_HEIGHT)
        || (profile->heading_near_y >= RAMP_WHITE_LINE_HEIGHT)
        || (profile->lateral_y >= RAMP_WHITE_LINE_HEIGHT))
        return 0u;
    if (left_boundary != NULL)
        for (i = 0u; i < RAMP_WHITE_LINE_HEIGHT; i++) left_boundary[i] = -1;
    if (right_boundary != NULL)
        for (i = 0u; i < RAMP_WHITE_LINE_HEIGHT; i++) right_boundary[i] = -1;

    ramp_build_candidates(gray, params, debug_edge);
    ramp_fit_side(RAMP_SIDE_LEFT, params, left_models);
    ramp_fit_side(RAMP_SIDE_RIGHT, params, right_models);
    for (i = 0u; i < RAMP_TOP_MODELS; i++)
    {
        if (left_models[i].valid != 0u) left_count++;
        if (right_models[i].valid != 0u) right_count++;
    }
    if (left_count == 0u) { result->reason = RAMP_WHITE_LINE_REASON_LEFT_LOST; return 0u; }
    if (right_count == 0u) { result->reason = RAMP_WHITE_LINE_REASON_RIGHT_LOST; return 0u; }
    close_surface = (profile->use_close_surface_rule != 0u)
        ? ramp_is_close_surface(gray) : 0u;

    for (i = 0u; i < RAMP_TOP_MODELS; i++)
    {
        const ramp_model_t *left = &left_models[i];
        if (left->valid == 0u) continue;
        for (j = 0u; j < RAMP_TOP_MODELS; j++)
        {
            const ramp_model_t *right = &right_models[j];
            int32 expansion;
            int32 denominator;
            int32 vanish_y_q8;
            int32 vanish_x_q8;
            int32 left_near_q8;
            int32 right_near_q8;
            int32 near_width_q8;
            int32 surface_quality;
            int32 pair_score;
            uint8 overlap_first;

```

```

uint8 left_span;
uint8 right_span;
uint8 left_limit;
uint8 right_limit;

if (right->valid == 0u) continue;
expansion = (int32)right->slope_q10 - left->slope_q10;
if (expansion <= 0) continue;
if ((profile->use_expansion_limit != 0u)
    && ((expansion < profile->min_expansion_q10)
        || (expansion > profile->max_expansion_q10))) continue;
denominator = (int32)left->slope_q10 - right->slope_q10;
if (denominator == 0) continue;
vanish_y_q8 = ((int32)(right->intercept_q8 - left->intercept_q8) * 1024)
    / denominator;
vanish_x_q8 = left->intercept_q8
    + ((int32)left->slope_q10 * vanish_y_q8) / 1024;
if ((profile->use_vanishing_limit != 0u)
    && ((vanish_y_q8 < profile->min_vanishing_y_q8)
        || (vanish_y_q8 > profile->max_vanishing_y_q8)
        || (vanish_x_q8 < profile->min_vanishing_x_q8)
        || (vanish_x_q8 > profile->max_vanishing_x_q8))) continue;
overlap_first = (left->first_y > right->first_y) ? left->first_y : right->first_y;
overlap_last = (left->last_y < right->last_y) ? left->last_y : right->last_y;
if (overlap_last < overlap_first) continue;
overlap_span = (uint8)(overlap_last - overlap_first);
if (overlap_span < ((close_surface != 0u)
    ? RAMP_CLOSE_MIN_OVERLAP_SPAN : profile->min_pair_overlap)) continue;
left_span = (uint8)(left->last_y - left->first_y);
right_span = (uint8)(right->last_y - right->first_y);
left_limit = (uint8)((2u * left_span < params->max_extrapolation)
    ? 2u * left_span : params->max_extrapolation);
right_limit = (uint8)((2u * right_span < params->max_extrapolation)
    ? 2u * right_span : params->max_extrapolation);
if ((profile->require_fixed_sample_support != 0u)
    && ((ramp_distance_to_interval(profile->heading_far_y, left) > left_limit)
        || (ramp_distance_to_interval(profile->heading_near_y, left) > left_limit)
        || (ramp_distance_to_interval(profile->lateral_y, left) > left_limit)
        || (ramp_distance_to_interval(profile->heading_far_y, right) > right_limit)
        || (ramp_distance_to_interval(profile->heading_near_y, right) > right_limit)
        || (ramp_distance_to_interval(profile->lateral_y, right) > right_limit))) continue;
left_near_q8 = ramp_x_q8(left, profile->lateral_y);
right_near_q8 = ramp_x_q8(right, profile->lateral_y);
near_width_q8 = right_near_q8 - left_near_q8;
if (near_width_q8 < profile->min_near_width_q8) continue;
if ((close_surface != 0u)
    && (near_width_q8 <= RAMP_CLOSE_MIN_NEAR_WIDTH_Q8)) continue;
if (near_width_q8 > 19251)
{
    if ((left->slope_q10 > -512) || (right->slope_q10 < 512)
        || (expansion < 1536) || (overlap_last >= 30u)) continue;
}
if (ramp_pair_surface_valid(gray, params, left, right, &surface_quality) == 0u)
    continue;
pair_score = left->score / left->support + right->score / right->support
    + (int32)overlap_span * 30 + (near_width_q8 * 5) / 256
    + surface_quality;
if (near_width_q8 > 19251) pair_score += expansion / 10;
if (pair_score > best_score)
{
    best_score = pair_score;
}

```

```

    }
}
}
if ((best_left == NULL) || (best_right == NULL))
{
    result->reason = RAMP_WHITE_LINE_REASON_NO_PAIR;
    return 0u;
}
result->left_slope_q10 = best_left->slope_q10;
result->right_slope_q10 = best_right->slope_q10;
result->left_intercept_q8 = best_left->intercept_q8;
result->right_intercept_q8 = best_right->intercept_q8;
{
    int32 far_center_q8 = (ramp_x_q8(best_left, profile->heading_far_y)
        + ramp_x_q8(best_right, profile->heading_far_y)) / 2;
    int32 heading_near_center_q8 =
        (ramp_x_q8(best_left, profile->heading_near_y)
        + ramp_x_q8(best_right, profile->heading_near_y)) / 2;
    int32 near_left_q8 = ramp_x_q8(best_left, profile->lateral_y);
    int32 near_right_q8 = ramp_x_q8(best_right, profile->lateral_y);
    int32 near_center_q8 = (near_left_q8 + near_right_q8) / 2;
    int32 width_q8 = near_right_q8 - near_left_q8;
    result->heading_error_q8 = ramp_saturate_i16(heading_near_center_q8 - far_center_q8);
    result->lateral_error_q12 = ramp_saturate_i16(
        ((RAMP_CENTER_Q8 - near_center_q8) * 4096) / width_q8);
}
result->left_confidence = best_left->confidence;
result->right_confidence = best_right->confidence;
result->valid = 1u;
result->reason = RAMP_WHITE_LINE_REASON_OK;
for (i = 0u; i < RAMP_WHITE_LINE_HEIGHT; i++)
{
    int32 lx = ramp_round_shift(ramp_x_q8(best_left, i), 8u);
    int32 rx = ramp_round_shift(ramp_x_q8(best_right, i), 8u);
    if ((left_boundary != NULL) && (lx >= 0) && (lx < (int32)RAMP_WHITE_LINE_WIDTH)
        && ((profile->render_supported_rows_only == 0u)
            || ((i >= best_left->first_y) && (i <= best_left->last_y))))
        left_boundary[i] = (int16)lx;
    if ((right_boundary != NULL) && (rx >= 0) && (rx < (int32)RAMP_WHITE_LINE_WIDTH)
        && ((profile->render_supported_rows_only == 0u)
            || ((i >= best_right->first_y) && (i <= best_right->last_y))))
        right_boundary[i] = (int16)rx;
    if (debug_edge != NULL)
    {
        if ((lx >= 0) && (lx < (int32)RAMP_WHITE_LINE_WIDTH)
            && ((profile->render_supported_rows_only == 0u)
                || ((i >= best_left->first_y) && (i <= best_left->last_y))))
            debug_edge[(uint16)i * RAMP_WHITE_LINE_WIDTH + (uint8)lx] = 255u;
        if ((rx >= 0) && (rx < (int32)RAMP_WHITE_LINE_WIDTH)
            && ((profile->render_supported_rows_only == 0u)
                || ((i >= best_right->first_y) && (i <= best_right->last_y))))
            debug_edge[(uint16)i * RAMP_WHITE_LINE_WIDTH + (uint8)rx] = 255u;
    }
}
}
return 1u;
}

uint8 ramp_white_line_detect(const uint8 *gray,
    const ramp_white_line_params_t *params,
    ramp_white_line_result_t *result,

```

```
        int16 left_boundary[RAMP_WHITE_LINE_HEIGHT],
        int16 right_boundary[RAMP_WHITE_LINE_HEIGHT],
        uint8 *debug_edge)
{
    static const ramp_white_line_profile_t profile = RAMP_WHITE_LINE_PROFILE_DEFAULT;

    return ramp_white_line_detect_profile(gray, params, &profile, result,
        left_boundary, right_boundary,
        debug_edge);
}
```

C.5 人工草地识别入口

文件： project/code/vision/grass_road.c

```

uint8 grass_road_detect(const uint8 *gray, const grass_road_params_t *params_in,
                        grass_road_result_t *result)
{
    grass_road_params_t params;
    grass_line_t best_left;
    grass_line_t best_right;
    int32 best_score = -1;
    uint8 threshold;
    uint8 seed_index;
    if ((gray == NULL) || (params_in == NULL) || (result == NULL)) return 0u;
    memset(result, 0, sizeof(*result));
    result->reason = GRASS_ROAD_REASON_INVALID_ARGUMENT;
    params = *params_in;
    params.dark_percent = grass_clamp_u8(params.dark_percent, 5u, 35u);
    params.inside_margin = grass_clamp_u8(params.inside_margin, 1u, 64u);
    params.min_gradient = grass_clamp_u8(params.min_gradient, 1u, 60u);
    params.min_support = grass_clamp_u8(params.min_support, 6u, 20u);
    params.track_window = grass_clamp_u8(params.track_window, 3u, 12u);
    params.refine_window = grass_clamp_u8(params.refine_window, 2u, 8u);
    params.refine_pad = grass_clamp_u8(params.refine_pad, 0u, 6u);
    threshold = grass_dark_threshold(gray, params.dark_percent);
    memset(&best_left, 0, sizeof(best_left));
    memset(&best_right, 0, sizeof(best_right));
    for (seed_index = 0u; seed_index < GRASS_SEED_COUNT; seed_index++)
    {
        grass_line_t left;
        grass_line_t right;
        uint8 far_y;
        uint8 near_y;
        int32 far_width;
        int32 near_width;
        int32 score;
        if ((grass_track_side(gray, 0u, grass_seed_rows[seed_index], threshold, &params, &left) == 0u)
            || (grass_track_side(gray, 1u, grass_seed_rows[seed_index], threshold, &params, &right) == 0u)
            || (left.slope_q10 >= -51) || (right.slope_q10 <= 51)) continue;
        if ((grass_refine_line(gray, 0u, threshold, &params, &left) == 0u)
            || (grass_refine_line(gray, 1u, threshold, &params, &right) == 0u)
            || (left.slope_q10 >= -51) || (right.slope_q10 <= 51)) continue;
        far_y = (left.first_y > right.first_y) ? left.first_y : right.first_y;
        near_y = (left.last_y < right.last_y) ? left.last_y : right.last_y;
        if ((near_y <= far_y) || ((uint8)(near_y - far_y) < 7u)) continue;
        far_width = grass_line_x_q8(&right, far_y) - grass_line_x_q8(&left, far_y);
        near_width = grass_line_x_q8(&right, near_y) - grass_line_x_q8(&left, near_y);
        if ((far_width < 3072) || (far_width > 14848) || (near_width < 6144)
            || (near_width > 24576) || (near_width - far_width < 768)) continue;
        score = (int32)(near_y - far_y) * 5 + left.support + right.support
            + (left.mean_evidence + right.mean_evidence) / 10;
        if (score > best_score)
        {
            best_score = score;
            best_left = left;
            best_right = right;
        }
    }
    if (best_score < 0)
    {
        if (grass_fallback_detect(gray, result) != 0u) return 1u;
        result->reason = GRASS_ROAD_REASON_NO_PAIR;
        return 0u;
    }
}

```

```

{
    uint8 sample_y = (best_left.last_y < best_right.last_y) ? best_left.last_y : best_right.last_y;
    int32 left_q8 = grass_line_x_q8(&best_left, sample_y);
    int32 right_q8 = grass_line_x_q8(&best_right, sample_y);
    int32 width_q8 = right_q8 - left_q8;
    result->left_slope_q10 = best_left.slope_q10;
    result->right_slope_q10 = best_right.slope_q10;
    result->left_intercept_q8 = best_left.intercept_q8;
    result->right_intercept_q8 = best_right.intercept_q8;
    result->heading_slope_q10 = (int16)((((int32)best_left.slope_q10 + best_right.slope_q10) / 2);
    result->lateral_error_q12 = grass_saturate_i16(
        ((GRASS_CENTER_Q8 - (left_q8 + right_q8) / 2) * 4096) / width_q8);
    result->left_confidence = (best_left.support * 255u > 49u * 255u)
        ? 255u : (uint8)(best_left.support * 255u / 49u);
    result->right_confidence = (best_right.support * 255u > 49u * 255u)
        ? 255u : (uint8)(best_right.support * 255u / 49u);
    result->valid = 1u;
    result->reason = GRASS_ROAD_REASON_OK;
}
return 1u;
}

```

C.6 视觉方向误差融合程序

文件：project/code/vision/vision_error_fusion.c

```

#include "vision_error_fusion.h"

#include <math.h>

static float vision_error_fusion_clamp(float value, float minimum, float maximum)
{
    if (value < minimum)
        return minimum;
    if (value > maximum)
        return maximum;
    return value;
}

static float vision_error_fusion_deadband(float error, float deadband)
{
    if (error > deadband)
        return error - deadband;
    if (error < -deadband)
        return error + deadband;
    return 0.0f;
}

float vision_error_fusion_turn_target(const vision_error_fusion_params_t *params,
                                     float heading_error,
                                     float lateral_error)
{
    float heading_effective;
    float lateral_effective;
    float lateral_abs;
    float lateral_weight;
    float target;

    if ((params == NULL)
        || !isfinite(heading_error)
        || !isfinite(lateral_error)
        || !isfinite(params->lateral_near)
        || !isfinite(params->lateral_far)
        || !isfinite(params->lateral_weight_min)
        || !isfinite(params->lateral_deadband)
        || !isfinite(params->gain)
        || !isfinite(params->turn_rate_limit_dps)
        || !isfinite(params->heading_scale)
        || !isfinite(params->lateral_scale)
        || (params->lateral_near < 0.0f)
        || (params->lateral_far <= params->lateral_near)
        || (params->gain < 0.0f)
        || (params->lateral_deadband < 0.0f)
        || (params->heading_scale < 0.0f)
        || (params->lateral_scale < 0.0f)
        || (params->turn_rate_limit_dps <= 0.0f))
    {
        return 0.0f;
    }

    heading_effective = params->heading_scale * heading_error;
    lateral_effective = params->lateral_scale
        * vision_error_fusion_deadband(lateral_error,
                                       params->lateral_deadband);
    if (!isfinite(heading_effective) || !isfinite(lateral_effective))
    {
        return 0.0f;
    }

```

```

lateral_weight = vision_error_fusion_clamp(
    (lateral_abs - params->lateral_near)
    / (params->lateral_far - params->lateral_near),
    0.0f, 1.0f);
lateral_weight = vision_error_fusion_clamp(params->lateral_weight_min,
    0.0f, 1.0f)
    + (1.0f - vision_error_fusion_clamp(params->lateral_weight_min,
    0.0f, 1.0f)) * lateral_weight;
target = params->gain
    * (lateral_weight * lateral_effective
    + (1.0f - lateral_weight) * heading_effective);
return vision_error_fusion_clamp(target,
    -params->turn_rate_limit_dps,
    params->turn_rate_limit_dps);
}

```